Functions in Python

Introduction to Functions

A **function** is a named block of code/sub program that performs a specific task. It helps in:

- **Code Reusability**: write once, use multiple times
- **Modularity**: break complex problems into smaller, manageable parts
- **Maintainability**: easier to debug and update

Types of Functions

- **Built-in Functions**
- **Functions defined in Modules**
- **User-Defined Functions**

1. Built-in Functions

- Pre-defined in Python.
- No import required.
- No definition required

```
print("Hello World") # Output: Hello World
x = max(10, 20, 5) # x = 20
y = len("Python") # y = 6
z = type(10) # z = <class 'int'>
```

Common built-ins: `print()`, `input()`, `len()`, `max()`, `min()`, `sum()`, `type()`, `int()`, `str()`, `float()`.

2. Functions defined in Modules

- Stored in external modules.
- Must be imported.

```
import math
print(math.sqrt(25)) # Output: 5.0
print(math.pow(2, 3)) # Output: 8.0

import random
print(random.randint(1, 10)) # random number 1-10
```

Common module functions: `math.sqrt()`, `random.randint()`, `datetime.datetime()`.

3. User-Defined Functions

Created by the programmer for specific requirements.
It has 2 parts:

- Function Definition
- Function Call

▹ Creating User-Defined Functions

Syntax - Function Definition

```
def function_name(parameters):
    # Function body
    return value # optional
```

Syntax - Function Call

```
greet(arguments) # Only the function name and argument
```

Example

```
def greet(name):
    print("Hello", name)

greet("Alice")
# Output: Hello Alice
```

▹ Arguments & Parameters

- **Formal Parameters**: variables listed in the function **definition** that receive values.
- **Actual Arguments**: actual values passed to the function during the **call**.

```
def add(a, b): # a and b are FORMAL PARAMETERS
    return a + b

result = add(5, 3) # 5 and 3 are ACTUAL ARGUMENTS
print(result) # Output: 8
```

▹ Formal Parameters (in Function Definition)

1. Positional Parameters

Regular parameters that receive values based on position/order.

Syntax:

```
def function_name(param1, param2, param3):
    # function body
```

Example:

```
def display_info(name, age, city): # Positional parameters
    print(f"Name: {name}, Age: {age}, City: {city}")

display_info("John", 25, "Delhi")
# Output: Name: John, Age: 25, City: Delhi
```

2. Default Parameters

Parameters with **predefined values** in the function definition. Used when no argument is passed.

Syntax:

```
def function_name(param1, param2="default_value"):
    # function body
```

Key Points:

- Default parameters are **optional** during function call
- If no argument is passed, default value is used
- If argument is passed, it **overrides** the default value
- Must be placed **after** non-default parameters

Example:

```
def greet(name, message="Good Morning"): # Default parameter
    print(message, name)

greet("Alice") # Uses default: Good Morning Alice
greet("Bob", "Good Evening") # Overrides default: Good Evening Bob
```

```
def power(base, exponent=2): # Default parameter
    return base ** exponent

print(power(5)) # Output: 25 (uses default exponent=2)
print(power(3, 4)) # Output: 81 (overrides default)
```

Important Rule: Default parameters must be placed **after** non-default parameters.

```
# CORRECT:
def func(a, b=10, c=20): pass

# INCORRECT (SyntaxError):
# def func(a=10, b, c=20): pass # Default can't come before non-
default
```

3. Variable-length Parameters (*args, **kwargs)

Parameters that can accept **any number** of arguments.

3.1 *args (Non-Keyword Variable Parameters)

Accepts any number of **positional** arguments as a **tuple**.

Syntax:

```
def function_name(*args):
    # args is a tuple containing all extra arguments
```

Key Points:

- The name `args` is conventional but not mandatory (the `*` is important)
- Arguments are collected as a **tuple**
- Can pass any number of positional arguments (0 or more)

Example:

```
def sum_all(*args): # Variable-length parameter
    total = 0
    for num in args:
        total += num
    return total

print(sum_all(1, 2, 3)) # Output: 6
print(sum_all(10, 20, 30, 40)) # Output: 100
print(sum_all(5)) # Output: 5
print(sum_all()) # Output: 0 (empty tuple)
```

3.2 **kwargs (Keyword Variable Parameters)

Accepts any number of **keyword** arguments as a **dictionary**.

Syntax:

```
def function_name(**kwargs):
    # kwargs is a dictionary of all extra keyword arguments
```

Key Points:

- The name `kwargs` is conventional (the `**` is important)
- Arguments are collected as a **dictionary**
- Can pass any number of keyword arguments (0 or more)

Example:

```
def display_student(**kwargs): # Variable-length parameter
   for key, value in kwargs.items():
      print(f"{key}: {value}")

display_student(name="Rahul", age=17, city="Delhi")
# Output:
# name: Rahul
# age: 17
# city: Delhi
```

3.3 Combining *args and **kwargs

Syntax:

```
def function_name(param1, param2="default", *args, **kwargs):
   # function body
```

Example:

```
def display_all(required_param, *args, **kwargs):
   print("Required:", required_param)
   print("*args:", args)
   print("**kwargs:", kwargs)

display_all("Hello", 10, 20, 30, name="Alice", age=25)
# Output:
# Required: Hello
# *args: (10, 20, 30)
# **kwargs: {'name': 'Alice', 'age': 25}
```

▹ Actual Arguments (in Function Call)

1. Positional Arguments

Arguments passed in the **same order** as parameters are defined.

Syntax:

```
function_name(value1, value2, value3)
```

Key Points:

- Order matters - first argument goes to first parameter
- Number of arguments must match number of parameters
- Most common way to pass arguments

Example:

```
def display_info(name, age, city):
    print(f"Name: {name}, Age: {age}, City: {city}")

display_info("John", 25, "Delhi") # Positional arguments
# Output: Name: John, Age: 25, City: Delhi
```

2. Keyword Arguments

Arguments passed using the **parameter names**.

Syntax:

```
function_name(param1=value1, param2=value2, param3=value3)
```

Key Points:

- Order **doesn't matter** - matched by parameter name
- Makes code more readable
- Must come **after** positional arguments if both are used

Example:

```
def student(name, age, course):
    print(f"{name} is {age} years old and studies {course}")
```

```
# Using keyword arguments (order doesn't matter)
student(name="Rahul", course="CS", age=17) # Keyword arguments
# Output: Rahul is 17 years old and studies CS

student(age=18, name="Priya", course="Math")
# Output: Priya is 18 years old and studies Math
```

Important: Keyword arguments must come **after** positional arguments.

```
# CORRECT:
student("Rahul", age=17, course="CS") # Positional then keyword

# INCORRECT (SyntaxError):
# student(name="Rahul", 17, "CS") # Keyword before positional
```

3. Variable-length Arguments (*args, **kwargs)

Passing any number of arguments during function call.

3.1 Passing *args

Syntax:

```
function_name(value1, value2, value3, ...) # Any number of
positional args
```

Example:

```
def sum_all(*args):
    return sum(args)

print(sum_all(1, 2, 3)) # Passing 3 arguments
print(sum_all(10, 20, 30, 40)) # Passing 4 arguments
print(sum_all()) # Passing 0 arguments
```

3.2 Passing **kwargs

Syntax:

```
function_name(key1=value1, key2=value2, ...) # Any number of keyword args
```

Example:

```
def display_student(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

display_student(name="Rahul", age=17, city="Delhi") # 3 keyword args
display_student(course="CS", grade="A") # 2 keyword args
```

Summary: Formal Parameters vs Actual Arguments

Category	Type	Where defined	Purpose
Formal Parameters	Positional Parameters	Function definition	Receive values by position
	Default Parameters	Function definition	Have predefined values
	Variable-length (*args, **kwargs)	Function definition	Accept any number of arguments
Actual Arguments	Positional Arguments	Function call	Pass values by position
	Keyword Arguments	Function call	Pass values by parameter name
	Variable-length (*args, **kwargs)	Function call	Pass any number of values

Important Order Rule: When combining different types in function definition:

1. Positional parameters
2. Default parameters
3. `*args` (variable positional)
4. `**kwargs` (variable keyword)

```
# Correct order in DEFINITION:
def correct_order(a, b=10, *args, **kwargs):
    pass

# Incorrect order (SyntaxError):
# def wrong_order(a, *args, b=10, **kwargs): # default after *args
# def wrong_order2(**kwargs, a, b): # **kwargs must be last
```

▹ Function Returning Value(s)

Use `return` statement. Without it, function returns `None`.

1. Single value

```
def square(num):
    return num * num

result = square(5)
print(result) # Output: 25
```

2. Multiple values (tuple)

```
def calculate(a, b):
    sum_val = a + b
    diff_val = a - b
    product_val = a * b
    return sum_val, diff_val, product_val

s, d, p = calculate(10, 5)
print(s, d, p) # Output: 15 5 50
```

3. No return → **None**

```python
def display():
    print("Hello World")

result = display() # Output: Hello World
print(result) # Output: None
```

▹ Flow of Execution

Order in which statements are executed:

- Python executes top-to-bottom.
- Function definitions are stored in memory when encountered.
- Function body executes **only when called**.
- Execution jumps to the function and returns to the calling point.

```python
print("1. Program starts")

def say_hello():
    print("3. Inside the function")
    print("4. Function completes")

print("2. Before function call")
say_hello()
print("5. After function call")
```

▶ Output:

```
1. Program starts
2. Before function call
3. Inside the function
4. Function completes
5. After function call
```

▹ Scope of Variables

Defines where a variable can be accessed.

📍 Local Scope

🌍 Global Scope

Variables defined inside a function are **local**.

```
def my_function():
    x = 10 # local variable
    print("Inside:", x)

my_function() # Inside: 10
# print(x) → NameError: x is not defined
```

Variables defined outside any function are **global**.

```
x = 100 # global
def func1():
    print(x) # reads global
def func2():
    x = 50 # local, hides global
    print(x) # 50
func1() # 100
func2() # 50
print(x) # 100 (global unchanged)
```

The `global` keyword

To modify a global variable inside a function, use `global`.

```
counter = 0 # global

def increment():
    global counter
    counter += 1
    print(counter)

increment() # Output: 1
increment() # Output: 2
print(counter) # Output: 2
```

> ⚠️ **Important:** Reading a global inside a function is allowed; to **modify** it, you must use global.

▹ Practice Questions

Q1. Even or Odd

```python
def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

print(check_even_odd(7)) # Odd
print(check_even_odd(10)) # Even
```

Q2. Default parameter (discount)

```python
def calculate_discount(price, discount=10):
    discounted_price = price - (price * discount / 100)
    return discounted_price

print(calculate_discount(1000)) # 900.0
print(calculate_discount(1000, 20)) # 800.0
```

Q3. Multiple return values (circle)

```python
def circle_info(radius):
    area = 3.14159 * radius * radius
    circumference = 2 * 3.14159 * radius
    return area, circumference

a, c = circle_info(5)
print(f"Area: {a:.2f}, Circumference: {c:.2f}")
# Area: 78.54, Circumference: 31.42
```

Q4. Global vs local

```python
total = 100

def update_total(value):
    global total
    total += value
    print("Updated total:", total)

update_total(50) # Updated total: 150
print(total) # 150
```

📋 Summary Table

Concept	Description	Example
Built-in Function	Pre-defined in Python	`print()`, `len()`
Module Function	Defined in imported modules	`math.sqrt()`, `random.randint()`
User-Defined Function	Created by programmer	`def greet():`
Parameters	Variables in function definition	`def add(a, b)`
Arguments	Values passed to function	`add(5, 3)`
Default Parameter	Parameter with default value	`def greet(name, msg="Hi")`
Positional Argument	Matched by position	`display(10, 20)`
Return Statement	Returns value from function	`return result`
Local Scope	Variables inside function	`x = 10` inside function
Global Scope	Variables outside function	`x = 10` outside function
`global` Keyword	Modify global variable	`global x`